

Project 1 Rubric – Vigenère Cipher

Keygen.cpp file

```
#include <vector>
#include <iostream>
#include <fstream>
#include <cstdlib>

//this file is for generating a random key of specified length and writing it to an
output file. Using /dev/urandom device to generate random bytes
int main(int command_arg, char* list_arg[]) { //command_arg = command line arguments
count, list_arg = array of command line arguments

    if (command_arg != 3) { //looking for command line arguments
        std::cerr << "Usage: ./keygen <length> <outputfile>" << std::endl;
        return 1; //return 1 if error occurs
    }

    int length = std::atoi(list_arg[1]);
    const char* filename = list_arg[2];

    if (length <= 0) { //checks for a positive key length
        std::cerr << "Key length must be positive." << std::endl;
        return 1;
    }

    std::ifstream urandom("/dev/urandom", std::ios::in | std::ios::binary);
    if (!urandom) { //checks if urandom is open right or not
        std::cerr << "Error opening /dev/urandom" << std::endl;
        return 1;
    }

    std::ofstream outfile(filename, std::ios::out | std::ios::binary);
    if (!outfile) {
        std::cerr << "Error opening output file." << std::endl;
```

```

        return 1;
    }

    for (int i = 0; i < length; i++) { //generating random bytes and writing it to the
output file
        char byte;
        urandom.read(&byte, 1);
        outfile.write(&byte, 1);
    }

    urandom.close();
    outfile.close();

    return 0;
}

```

Encrypt.cpp

```

#include <iostream>
#include <fstream>
#include <vector>
#include <cstdlib>

//this file is encrypting a plaintext file using a key file. It reads the key bytes
into a vector and then processes the plaintext file byte by byte.
int main(int command_arg, char* list_arg[]) {
    if (command_arg != 3) { //looking for command line arguments
        std::cerr << "Usage: ./encrypt <plaintext_file> <key_file>" << std::endl;
        return 1;
    }

    const char* plaintext_filename = list_arg[1];
    const char* key_filename = list_arg[2]; //I put this to make two files and they're
pointers storing the address

    std::ifstream key_file(key_filename, std::ios::in | std::ios::binary);
    if (!key_file) { //checking if key file is open right or not
        std::cerr << "Problem accessing key file." << std::endl;
        return 1;
    }
}

```

```

std::vector<unsigned char> key_bytes;//dynamic array to store key bytes
char byte;
while (key_file.read(&byte, 1)) {
    key_bytes.push_back(static_cast<unsigned char>(byte));
}
key_file.close();

if (key_bytes.empty()) { //checks if key file is empty
    std::cerr << "Key file is empty" << std::endl;
    return 1;
}

std::ifstream plaintext_file(plaintext_filename, std::ios::in | std::ios::binary);
if (!plaintext_file) { //checking if plaintext file is open right or not
    std::cerr << "Problem accessing  plaintext file." << std::endl;
    return 1;
}

size_t key_len = key_bytes.size();//storing the length of the key file
size_t key_index = 0;//keeping track of position

while (plaintext_file.read(&byte, 1)) { //processing the plaintext file byte by byte
    unsigned char plain_byte = static_cast<unsigned char>(byte);
    unsigned char cipher_byte = (plain_byte + key_bytes[key_index]) % 256;

    std::cout.write(reinterpret_cast<char*>(&cipher_byte), 1);

    key_index = (key_index + 1) % key_len;//mod athrimitc
}

plaintext_file.close();//
return 0;
}

```

Decrypt.cpp

```

#include <iostream>
#include <fstream>
#include <vector>
#include <cstdlib>

```

```

//this file is decrypting a ciphertext file using a key file. It reads the key bytes
into a vector and then processes the ciphertext file byte by byte.
int main(int command_arg, char* list_arg[]) {

    if (command_arg != 3) { //looking for command line arguments
        std::cerr << "Usage: ./decrypt <ciphertext_file> <key_file>" << std::endl;
        return 1;
    }

    const char* ciphertext_filename = list_arg[1]; //I put this to make two files and
they're pointers storing the address
    const char* key_filename = list_arg[2];
    std::ifstream key_file(key_filename, std::ios::in | std::ios::binary);
    if (!key_file) { //checking if key file is open right or not
        std::cerr << "Error opening key file." << std::endl;
        return 1;
    }

    std::vector<unsigned char> key_bytes; //dynamic array to store key bytes
    char byte;
    while (key_file.read(&byte, 1)) {
        key_bytes.push_back(static_cast<unsigned char>(byte));
    }
    key_file.close();

    if (key_bytes.empty()) { //checks if key file is empty
        std::cerr << "Key file is empty!" << std::endl;
        return 1;
    }

    std::ifstream ciphertext_file(ciphertext_filename, std::ios::in |
std::ios::binary);
    if (!ciphertext_file) {
        std::cerr << "Error opening ciphertext file." << std::endl;
        return 1;
    }

    size_t key_len = key_bytes.size();
    size_t key_index = 0;

    while (ciphertext_file.read(&byte, 1)) { //processing the ciphertext file byte by
byte
        unsigned char cipher_byte = static_cast<unsigned char>(byte);

```

```

        unsigned char plain_byte = (cipher_byte + 256 - key_bytes[key_index]) %
256;//mod athrimitc to get the original byte value

        std::cout.write(reinterpret_cast<char*>(&plain_byte), 1);

        key_index = (key_index + 1) % key_len;//mod athrimitc to move to the next key
byte
    }

    ciphertext_file.close();
    return 0;
}

```

Result from my Encryption and Decryption files:

```

michaeljordan@Michaels-MacBook-Pro-7 mj_vigenere_project % xxd ciphertextfile

[michaeljordan@Michaels-MacBook-Pro-7 mj_vigenere_project % cat plaintext
[michaeljordan@Michaels-MacBook-Pro-7 mj_vigenere_project % cat plaintext
[michaeljordan@Michaels-MacBook-Pro-7 mj_vigenere_project % nano plaintext.txt
[michaeljordan@Michaels-MacBook-Pro-7 mj_vigenere_project % cat plaintext.txt
Im really trying to see if this code works
[michaeljordan@Michaels-MacBook-Pro-7 mj_vigenere_project % ./encrypt plaintext.txt keyfile > ciphertextfile

michaeljordan@Michaels-MacBook-Pro-7 mj_vigenere_project % xxd ciphertextfile

00000000: 2af5 01fa 46e9 4df4 5aa8 55fa 5af1 4fef  *...F.M.Z.U.Z.O.
00000010: 01fc 50a8 54ed 46a8 4aee 01fc 49f1 54a8  ..P.T.F.J...I.T.
00000020: 44f7 45ed 01ff 50fa 4cfb 0192          D.E...P.L...
[michaeljordan@Michaels-MacBook-Pro-7 mj_vigenere_project % ./decrypt ciphertextfile keyfile > recovered_plaintext
[michaeljordan@Michaels-MacBook-Pro-7 mj_vigenere_project % ./decrypt ciphertextfile keyfile > recovered_plaintext.txt
[michaeljordan@Michaels-MacBook-Pro-7 mj_vigenere_project % cat recovered_plaintext.txt
Im really trying to see if this code works
michaeljordan@Michaels-MacBook-Pro-7 mj_vigenere_project % █

```

Decrypting mystery.enc:

1. The length of the key used to encrypt **mystery.enc**.

The key length used to encrypt mystery.enc is 37. I found this by making a code that calculates the Index of Coincidence for a group of bytes. From reading the article I know to look for a spike where if given a value of around 0.067 it's encrypted with a monoalphabetic method.

```

crack      decrypt      encrypt      keygen      mystery.enc      plaintext.txt      test.txt
[michaeljordan@Michaels-MacBook-Pro-7 mj_vigenere_project % ./crack
Key Length: 1 Avg IoC: 3.28832
Key Length: 2 Avg IoC: -0.572598
Key Length: 3 Avg IoC: -1.38698
Key Length: 4 Avg IoC: -0.572587
Key Length: 5 Avg IoC: 0.0792892
Key Length: 6 Avg IoC: -0.108483
Key Length: 7 Avg IoC: 0.0732258
Key Length: 8 Avg IoC: -0.572582
Key Length: 9 Avg IoC: -0.0521268
Key Length: 10 Avg IoC: -0.02586
Key Length: 11 Avg IoC: -0.0166879
Key Length: 12 Avg IoC: 0.0156589
Key Length: 13 Avg IoC: 0.0156581
Key Length: 14 Avg IoC: 0.0156492
Key Length: 15 Avg IoC: 0.0156493
Key Length: 16 Avg IoC: 0.0156512
Key Length: 17 Avg IoC: 0.0156497
Key Length: 18 Avg IoC: 0.01565
Key Length: 19 Avg IoC: 0.0156617
Key Length: 20 Avg IoC: 0.0156493
Key Length: 21 Avg IoC: 0.0156472
Key Length: 22 Avg IoC: 0.0156483
Key Length: 23 Avg IoC: 0.0156489
Key Length: 24 Avg IoC: 0.0156583
Key Length: 25 Avg IoC: 0.0156487
Key Length: 26 Avg IoC: 0.0156583
Key Length: 27 Avg IoC: 0.0156482
Key Length: 28 Avg IoC: 0.0156484
Key Length: 29 Avg IoC: 0.0156492
Key Length: 30 Avg IoC: 0.0156499
Key Length: 31 Avg IoC: 0.0156508
Key Length: 32 Avg IoC: 0.0156508
Key Length: 33 Avg IoC: 0.0156497
Key Length: 34 Avg IoC: 0.0156464
Key Length: 35 Avg IoC: 0.0156478
Key Length: 36 Avg IoC: 0.0156481
Key Length: 37 Avg IoC: 0.0025865
Key Length: 38 Avg IoC: 0.0156515
Key Length: 39 Avg IoC: 0.0156466
Key Length: 40 Avg IoC: 0.0156478
Key Length: 41 Avg IoC: 0.0156479
Key Length: 42 Avg IoC: 0.0156436
Key Length: 43 Avg IoC: 0.0156466
Key Length: 44 Avg IoC: 0.0156462
Key Length: 45 Avg IoC: 0.0156462
Key Length: 46 Avg IoC: 0.0156466
Key Length: 47 Avg IoC: 0.0156468
Key Length: 48 Avg IoC: 0.0156472
Key Length: 49 Avg IoC: 0.0156516
Key Length: 50 Avg IoC: 0.0156449
Key Length: 51 Avg IoC: 0.0156487
Key Length: 52 Avg IoC: 0.0156473
Key Length: 53 Avg IoC: 0.015647
Key Length: 54 Avg IoC: 0.0156463
Key Length: 55 Avg IoC: 0.0156459
Key Length: 56 Avg IoC: 0.0156475
Key Length: 57 Avg IoC: 0.0156522
Key Length: 58 Avg IoC: 0.0156461
Key Length: 59 Avg IoC: 0.0156456
Key Length: 60 Avg IoC: 0.015645
[michaeljordan@Michaels-MacBook-Pro-7 mj_vigenere_project % ]

```

2. The name of the book is encrypted:

```

Key Length: 60 Avg IoC: 0.015645
[michaeljordan@Michaels-MacBook-Pro-7 mj_vigenere_project % touch cracking_37.cpp
[michaeljordan@Michaels-MacBook-Pro-7 mj_vigenere_project % nano cracking_37.cpp
[michaeljordan@Michaels-MacBook-Pro-7 mj_vigenere_project % clang++ cracking_37.cpp -o cracking_37
[michaeljordan@Michaels-MacBook-Pro-7 mj_vigenere_project % ./cracking_37
Recovered Key:
Two months after they weighed anchor

Decryption complete. Output written to cracked.txt
michaeljordan@Michaels-MacBook-Pro-7 mj_vigenere_project %

```

Original cracking.cpp file in Mac terminal:

```
UW PICO 5.87 File: cracking.cpp Modified

#include <iostream>
#include <fstream>
#include <vector>

//This code is for cracking the md file prof gave me

// Function to calculate Index of Coincidence for a group of bytes
double cal_ioc(const std::vector<unsigned char>& group) {
    // Array to count frequency of each possible byte (0-255)
    int freq[256] = {0};

    // Count how many times each byte appears
    for (unsigned char byte : group) {
        freq[byte]++;
    }

    // Total number of bytes in this group
    int N = group.size();

    // If group too small, IoC doesn't make sense
    if (N <= 1) return 0.0;

    double sum = 0.0;

    // Apply IoC formula:
    // sum(freq[i] * (freq[i] - 1))
    for (int i = 0; i < 256; i++) {
        sum += freq[i] * (freq[i] - 1);
    }

    // Divide by N * (N - 1)
    return sum / (double)(N * (N - 1));
}

int main() {
    // Open the encrypted file in binary mode
    std::ifstream file("mystery.enc", std::ios::binary);

    // Check if file opened successfully
    if (!file) {
        std::cout << "Error opening mystery.enc\n";
        return 1; // exit with error
    }

    // Vector to store all ciphertext bytes
    std::vector<unsigned char> ciphertext;

    unsigned char byte;

    // Read file byte-by-byte and store in vector
    while (file.read(&byte, 1)) {
        ciphertext.push_back(byte);
    }

    file.close();
}
```

Get Help

Exit

WriteOut

Justify

Read File

Where is

Prev Pg

Next Pg

Cut Text

UnCut Text

Cur Pos

To Spell

Errors and Coding Problems I ran into while constructing in my terminal:

```
Last login: Fri Feb 6 12:26:17 on tty8091
michaeljordan@Michaels-MacBook-Pro-7 ~ % mkdir mj_vigenere_project
michaeljordan@Michaels-MacBook-Pro-7 ~ % cd mj_vigenere_project
michaeljordan@Michaels-MacBook-Pro-7 mj_vigenere_project % touch keygen.cpp encrypt.cpp decrypt.cpp

michaeljordan@Michaels-MacBook-Pro-7 mj_vigenere_project % ls
decrypt.cpp  encrypt.cpp  keygen.cpp
michaeljordan@Michaels-MacBook-Pro-7 mj_vigenere_project % bash compile_everything.sh

bash: compile_everything.sh: No such file or directory
michaeljordan@Michaels-MacBook-Pro-7 mj_vigenere_project % touch compile_everything.sh

michaeljordan@Michaels-MacBook-Pro-7 mj_vigenere_project % nano compile_everything.sh

michaeljordan@Michaels-MacBook-Pro-7 mj_vigenere_project % chmod u+x compile_everything.sh

michaeljordan@Michaels-MacBook-Pro-7 mj_vigenere_project % bash compile_everything.sh

Undefined symbols for architecture arm64:
  "_main", referenced from:
    <initial-undefined>
ld: symbol(s) not found for architecture arm64
clang++: error: linker command failed with exit code 1 (use -v to see invocation)
Undefined symbols for architecture arm64:
  "_main", referenced from:
    <initial-undefined>
ld: symbol(s) not found for architecture arm64
clang++: error: linker command failed with exit code 1 (use -v to see invocation)
Undefined symbols for architecture arm64:
  "_main", referenced from:
    <initial-undefined>
ld: symbol(s) not found for architecture arm64
clang++: error: linker command failed with exit code 1 (use -v to see invocation)
michaeljordan@Michaels-MacBook-Pro-7 mj_vigenere_project % ls
compile_everything.sh  decrypt.cpp  encrypt.cpp  keygen.cpp
michaeljordan@Michaels-MacBook-Pro-7 mj_vigenere_project % bash compile_everything.sh

Undefined symbols for architecture arm64:
  "_main", referenced from:
    <initial-undefined>
ld: symbol(s) not found for architecture arm64
clang++: error: linker command failed with exit code 1 (use -v to see invocation)
Undefined symbols for architecture arm64:
  "_main", referenced from:
    <initial-undefined>
ld: symbol(s) not found for architecture arm64
clang++: error: linker command failed with exit code 1 (use -v to see invocation)
michaeljordan@Michaels-MacBook-Pro-7 mj_vigenere_project % nano keygen.cpp
michaeljordan@Michaels-MacBook-Pro-7 mj_vigenere_project % bash compile_everything.sh

Undefined symbols for architecture arm64:
  "_main", referenced from:
    <initial-undefined>
ld: symbol(s) not found for architecture arm64
clang++: error: linker command failed with exit code 1 (use -v to see invocation)
Undefined symbols for architecture arm64:
  "_main", referenced from:
    <initial-undefined>
```